

# Week 1, Day 1-2: SQL Server Basics

## CREATE, INSERT, UPDATE, DELETE, SELECT, WHERE, ORDER BY

---

### Table of Contents

1. [Introduction to SQL Server](#)
  2. [CREATE TABLE](#)
  3. [INSERT - Adding Data](#)
  4. [SELECT - Retrieving Data](#)
  5. [WHERE Clause - Filtering Data](#)
  6. [UPDATE - Modifying Data](#)
  7. [DELETE - Removing Data](#)
  8. [ORDER BY - Sorting Data](#)
  9. [Practice Exercises](#)
  10. [Interview Q&A](#)
- 

## Introduction to SQL Server

### What is SQL?

**SQL** (Structured Query Language) is used to communicate with databases. It's the standard language for managing relational databases.

### What is SQL Server?

**SQL Server** is a Relational Database Management System (RDBMS) developed by Microsoft. It stores data in structured tables with rows and columns.

### Key Concepts:

- **Database:** Container for related tables
- **Table:** Organized data in rows and columns
- **Row/Record:** A single entry in a table
- **Column/Field:** A specific attribute of the data
- **Primary Key:** Unique identifier for each row

### Real-World Example:

```
Students Table:
+---+-----+-----+-----+
| ID | Name  | Age  | Email |
+---+-----+-----+-----+
| 1  | Raj   | 20   | raj@.. |
| 2  | Priya | 21   | priya...|
+---+-----+-----+-----+
      ↑ PK      ↑      ↑
    Unique Column Column
```

---

---

## CREATE TABLE - Creating Tables

### Syntax:

```
CREATE TABLE TableName (  
    ColumnName DataType Constraints,  
    ColumnName DataType Constraints  
);
```

### Common Data Types:

| Type         | Description     | Example               |
|--------------|-----------------|-----------------------|
| INT          | Integer         | 25, -10, 0            |
| VARCHAR(n)   | Variable text   | 'John', 'Hello World' |
| NVARCHAR(n)  | Unicode text    | '👍', 'Ñoño'           |
| DATE         | Date only       | 2025-05-25            |
| DATETIME     | Date & Time     | 2025-05-25 10:30:00   |
| DECIMAL(p,s) | Decimal numbers | 1234.56               |
| BIT          | Boolean (0/1)   | 0, 1                  |
| FLOAT        | Floating point  | 3.14, 2.71            |

### Common Constraints:

| Constraint  | Purpose                       | Example                             |
|-------------|-------------------------------|-------------------------------------|
| PRIMARY KEY | Unique identifier             | ID INT PRIMARY KEY                  |
| NOT NULL    | Must have value               | Name VARCHAR(50) NOT NULL           |
| UNIQUE      | All values must be unique     | Email VARCHAR(100) UNIQUE           |
| DEFAULT     | Default value if not provided | Status VARCHAR(20) DEFAULT 'Active' |
| CHECK       | Validates condition           | Age INT CHECK (Age >= 18)           |

### Example 1: Students Table

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY IDENTITY(1,1),  
    FirstName NVARCHAR(50) NOT NULL,  
    LastName NVARCHAR(50) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    DateOfBirth DATE,  
    GPA DECIMAL(3,2),  
    IsActive BIT DEFAULT 1,  
    EnrollmentDate DATETIME DEFAULT GETDATE()  
);
```

**Breakdown:** - StudentID INT PRIMARY KEY IDENTITY(1,1) → Auto-increment starting from 1 - FirstName NVARCHAR(50) NOT NULL → Required field, max 50 characters - Email VARCHAR(100) UNIQUE → No duplicates allowed - GPA DECIMAL(3,2) → 3 total digits, 2 after decimal

(99.99 max) - IsActive BIT DEFAULT 1 → Default value is 1 (true/active)  
- EnrollmentDate DATETIME DEFAULT GETDATE() → Auto-sets current date/time

## Example 2: Courses Table

```
CREATE TABLE Courses (  
    CourseID INT PRIMARY KEY IDENTITY(1,1),  
    CourseName NVARCHAR(100) NOT NULL,  
    Credits INT CHECK (Credits > 0),  
    Instructor NVARCHAR(50),  
    Duration INT DEFAULT 4 -- in weeks  
);
```

---

## INSERT - Adding Data to Tables

### Syntax:

```
INSERT INTO TableName (Column1, Column2, Column3)  
VALUES (Value1, Value2, Value3);
```

### Key Points:

- Column order in INSERT must match VALUES order
- You can skip columns if they have DEFAULT or IDENTITY
- String values must be in single quotes: 'value'
- Dates must be in format: 'YYYY-MM-DD' or YYYY-MM-DD

## Example 1: Insert Single Student

```
INSERT INTO Students (FirstName, LastName, Email, DateOfBirth, GPA)  
VALUES ('Raj', 'Kumar', 'raj@email.com', '2004-03-15', 3.8);
```

**What happens:** - StudentID is auto-generated (IDENTITY) - IsActive defaults to 1 - EnrollmentDate defaults to current date/time

## Example 2: Insert Multiple Students (One Query)

```
INSERT INTO Students (FirstName, LastName, Email, DateOfBirth, GPA)  
VALUES  
    ('Priya', 'Singh', 'priya@email.com', '2004-05-20', 3.9),  
    ('Arjun', 'Patel', 'arjun@email.com', '2005-01-10', 3.5),  
    ('Neha', 'Sharma', 'neha@email.com', '2004-11-25', 3.7);
```

## Example 3: Insert Courses

```
INSERT INTO Courses (CourseName, Credits, Instructor)  
VALUES  
    ('.NET Full Stack', 4, 'Mr. Sharma'),  
    ('Web Development', 4, 'Ms. Priya'),  
    ('Database Design', 3, 'Mr. Kumar');
```

### ⚠ Common Mistakes:

```
-- WRONG - Mismatched columns and values  
INSERT INTO Students (FirstName, LastName)  
VALUES ('Raj', 'Kumar', 'raj@email.com'); -- 3 values for 2  
columns!  
  
-- CORRECT
```

```
INSERT INTO Students (FirstName, LastName, Email)
VALUES ('Raj', 'Kumar', 'raj@email.com');
```

---

## SELECT - Retrieving Data

### Syntax:

```
SELECT Column1, Column2 FROM TableName;
```

### Basic Examples:

#### Example 1: Select All Columns

```
SELECT * FROM Students;
```

#### Output:

| StudentID | FirstName | LastName | Email           | DateOfBirth |
|-----------|-----------|----------|-----------------|-------------|
| 1         | Raj       | Kumar    | raj@email.com   | 2004-03-15  |
| 2         | Priya     | Singh    | priya@email.com | 2004-05-20  |
| 3         | Arjun     | Patel    | arjun@email.com | 2005-01-10  |

#### Example 2: Select Specific Columns

```
SELECT FirstName, LastName, Email FROM Students;
```

#### Output:

| FirstName | LastName | Email           |
|-----------|----------|-----------------|
| Raj       | Kumar    | raj@email.com   |
| Priya     | Singh    | priya@email.com |
| Arjun     | Patel    | arjun@email.com |

#### Example 3: Select with Alias (Rename Column)

```
SELECT
    FirstName AS 'First Name',
    LastName AS 'Last Name',
    GPA AS 'Grade Point Average'
FROM Students;
```

#### Example 4: Select with Calculation

```
SELECT
    FirstName,
    LastName,
    GPA,
    GPA * 100 AS 'GPA Percentage'
FROM Students;
```

#### Output:

| FirstName | LastName | GPA  | GPA Percentage |
|-----------|----------|------|----------------|
| Raj       | Kumar    | 3.80 | 380            |
| Priya     | Singh    | 3.90 | 390            |

---

# WHERE Clause - Filtering Data

## Syntax:

```
SELECT Column1, Column2 FROM TableName WHERE Condition;
```

## Comparison Operators:

| Operator    | Meaning               |
|-------------|-----------------------|
| =           | Equal to              |
| != or <>    | Not equal to          |
| >           | Greater than          |
| <           | Less than             |
| >=          | Greater than or equal |
| <=          | Less than or equal    |
| LIKE        | Pattern matching      |
| IN          | Multiple values       |
| BETWEEN     | Range                 |
| IS NULL     | No value              |
| IS NOT NULL | Has value             |

### Example 1: Equal To

```
SELECT * FROM Students WHERE FirstName = 'Raj';
```

**Output:** Returns only Raj's record

### Example 2: Greater Than

```
SELECT FirstName, LastName, GPA FROM Students WHERE GPA > 3.7;
```

**Output:**

|           |          |      |
|-----------|----------|------|
| FirstName | LastName | GPA  |
| Raj       | Kumar    | 3.80 |
| Priya     | Singh    | 3.90 |

### Example 3: LIKE - Pattern Matching

```
SELECT * FROM Students WHERE Email LIKE '%@email.com';
```

**Explanation:** - % = wildcard (any characters) - \_ = single character wildcard

**More Examples:**

```
-- Find names starting with 'R'
SELECT * FROM Students WHERE FirstName LIKE 'R%';

-- Find names ending with 'a'
SELECT * FROM Students WHERE FirstName LIKE '%a';

-- Find names with 'rj' anywhere
SELECT * FROM Students WHERE FirstName LIKE '%rj%';
```

### Example 4: IN - Multiple Values

```
SELECT * FROM Students
WHERE FirstName IN ('Raj', 'Priya', 'Neha');
```

**Equivalent to:**

```
SELECT * FROM Students
WHERE FirstName = 'Raj' OR FirstName = 'Priya' OR FirstName =
'Neha';
```

### Example 5: BETWEEN - Range

```
SELECT * FROM Students WHERE GPA BETWEEN 3.5 AND 3.8;
```

**Output:** Students with GPA from 3.5 to 3.8 (inclusive)

### Example 6: IS NULL / IS NOT NULL

```
-- Find students without email
SELECT * FROM Students WHERE Email IS NULL;

-- Find students with email
SELECT * FROM Students WHERE Email IS NOT NULL;
```

### Example 7: AND / OR - Logical Operators

```
-- AND: Both conditions must be true
SELECT * FROM Students
WHERE GPA > 3.5 AND FirstName LIKE 'P%';

-- OR: At least one condition must be true
SELECT * FROM Students
WHERE GPA > 3.8 OR FirstName = 'Arjun';
```

### Example 8: NOT - Negation

```
SELECT * FROM Students WHERE NOT (GPA < 3.5);
```

**Equivalent to:**

```
SELECT * FROM Students WHERE GPA >= 3.5;
```

---

## ⇒ UPDATE - Modifying Existing Data

**Syntax:**

```
UPDATE TableName
SET Column1 = Value1, Column2 = Value2
WHERE Condition;
```

⚠ **CRITICAL: Always use WHERE clause!**

**Without WHERE, ALL rows will be updated!**

### Example 1: Update Single Column

```
UPDATE Students
SET GPA = 3.85
WHERE FirstName = 'Raj';
```

**Result:** Only Raj's GPA is changed to 3.85

### Example 2: Update Multiple Columns

```
UPDATE Students
SET
    GPA = 3.75,
    IsActive = 1
WHERE StudentID = 2;
```

### Example 3: Update with Calculation

```
-- Increase all GPAs by 0.1
UPDATE Students
SET GPA = GPA + 0.1
WHERE GPA < 4.0;
```

### Example 4: Update Based on Condition

```
-- Set status to inactive for low GPA
UPDATE Students
SET IsActive = 0
WHERE GPA < 2.0;
```

### DANGEROUS Example (Don't Do This!)

```
UPDATE Students SET GPA = 4.0;
-- ⚠ This updates ALL students! No WHERE clause!
```

---

## DELETE - Removing Data

### Syntax:

```
DELETE FROM TableName WHERE Condition;
```

⚠ **CRITICAL: Always use WHERE clause!**

**Without WHERE, ALL rows will be deleted!**

### Example 1: Delete Single Record

```
DELETE FROM Students WHERE StudentID = 5;
```

**Result:** Student with ID 5 is deleted

### Example 2: Delete Multiple Records

```
DELETE FROM Students WHERE GPA < 2.0;
```

**Result:** All students with GPA below 2.0 are deleted

### Example 3: Delete Inactive Students

```
DELETE FROM Students WHERE IsActive = 0;
```

### DANGEROUS Example (Don't Do This!)

```
DELETE FROM Students;
-- ⚠ This deletes ALL students! No WHERE clause!
```

### Safe Deletion Practice:

```
-- Step 1: Always SELECT first to verify what you're deleting
SELECT * FROM Students WHERE GPA < 2.0;

-- Step 2: If correct, then DELETE
DELETE FROM Students WHERE GPA < 2.0;
```

---

## ORDER BY - Sorting Data

### Syntax:

```
SELECT * FROM TableName
ORDER BY Column1 ASC/DESC, Column2 ASC/DESC;
```

### Key Points:

- ASC = Ascending (A to Z, 0 to 9) - **Default**
- DESC = Descending (Z to A, 9 to 0)
- Can sort by multiple columns

### Example 1: Ascending (Default)

```
SELECT FirstName, LastName, GPA FROM Students
ORDER BY GPA;
```

#### Output (Lowest to Highest GPA):

| FirstName | LastName | GPA  |
|-----------|----------|------|
| Arjun     | Patel    | 3.50 |
| Raj       | Kumar    | 3.80 |
| Priya     | Singh    | 3.90 |

### Example 2: Descending

```
SELECT FirstName, LastName, GPA FROM Students
ORDER BY GPA DESC;
```

#### Output (Highest to Lowest GPA):

| FirstName | LastName | GPA  |
|-----------|----------|------|
| Priya     | Singh    | 3.90 |
| Raj       | Kumar    | 3.80 |
| Arjun     | Patel    | 3.50 |

### Example 3: Sort by Text

```
SELECT FirstName, LastName FROM Students
ORDER BY FirstName ASC;
```

#### Output (Alphabetical):

| FirstName | LastName |
|-----------|----------|
| Arjun     | Patel    |
| Priya     | Singh    |
| Raj       | Kumar    |

### Example 4: Multiple Column Sort

```
SELECT FirstName, LastName, GPA FROM Students
ORDER BY GPA DESC, FirstName ASC;
```



**Explanation:** 1. First, sort by GPA (Highest first) 2. If GPA is same, then sort by FirstName (A to Z)

### Example 5: Combining WHERE and ORDER BY

```
SELECT FirstName, LastName, GPA FROM Students
WHERE GPA > 3.5
ORDER BY GPA DESC;
```

**Output:**

| FirstName | LastName | GPA  |
|-----------|----------|------|
| Priya     | Singh    | 3.90 |
| Raj       | Kumar    | 3.80 |
| Arjun     | Patel    | 3.50 |

---

## Practice Exercises

### Setup: Create and Populate Tables

```
-- Create Students Table
CREATE TABLE Students (
    StudentID INT PRIMARY KEY IDENTITY(1,1),
    FirstName NVARCHAR(50) NOT NULL,
    LastName NVARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    DateOfBirth DATE,
    GPA DECIMAL(3,2),
    IsActive BIT DEFAULT 1
);

-- Insert Sample Data
INSERT INTO Students (FirstName, LastName, Email, DateOfBirth, GPA)
VALUES
    ('Raj', 'Kumar', 'raj@email.com', '2004-03-15', 3.80),
    ('Priya', 'Singh', 'priya@email.com', '2004-05-20', 3.90),
    ('Arjun', 'Patel', 'arjun@email.com', '2005-01-10', 3.50),
    ('Neha', 'Sharma', 'neha@email.com', '2004-11-25', 3.70),
    ('Vikram', 'Verma', 'vikram@email.com', '2005-07-08', 2.80);
```

### Exercise 1: Basic SELECT

**Question:** Write a query to display all students with their FirstName, LastName, and GPA.

► [Click for Answer](#)

---

### Exercise 2: WHERE Clause

**Question:** Find all students with GPA greater than 3.5.

► [Click for Answer](#)

---

### Exercise 3: WHERE with LIKE

**Question:** Find all students whose FirstName starts with 'P'.

► [Click for Answer](#)

---

## Exercise 4: ORDER BY

**Question:** Display all students sorted by GPA in descending order.

► [Click for Answer](#)

---

## Exercise 5: WHERE + ORDER BY

**Question:** Find students with GPA  $\geq 3.6$  and sort by LastName alphabetically.

► [Click for Answer](#)

---

## Exercise 6: UPDATE

**Question:** Update Raj's GPA to 3.95.

► [Click for Answer](#)

---

## Exercise 7: DELETE

**Question:** Delete the student with ID 5 (Vikram).

► [Click for Answer](#)

---

## Exercise 8: Combined Query

**Question:** Find all active students with GPA between 3.5 and 3.9, sorted by GPA descending.

► [Click for Answer](#)

---

# Interview Q&A

## Q1: What is the difference between DELETE and TRUNCATE?

**Answer:** | Aspect | DELETE | TRUNCATE | |---|---|---| | **Speed** | Slower (row by row) | Faster (removes all at once) | | **WHERE** | Can use WHERE clause | Cannot use WHERE | | **Identity** | Doesn't reset IDENTITY | Resets IDENTITY to seed | | **Log** | Generates log entries | Minimal logging | | **Usage** | Remove specific rows | Remove all rows quickly |

```
-- DELETE - Can be selective
DELETE FROM Students WHERE GPA < 2.0;

-- TRUNCATE - Removes all rows
TRUNCATE TABLE Students;
```

---

## Q2: What is PRIMARY KEY and why do we need it?

**Answer:** - **PRIMARY KEY** is a unique identifier for each row - **Why needed:** - Ensures no duplicate records - Allows quick data retrieval - Maintains data integrity - Required for relationships between tables

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY, -- Unique identifier
    FirstName NVARCHAR(50)
);
```

---

### Q3: What is the difference between NULL and Empty String ('')?

**Answer:**

```
-- NULL = No value at all
INSERT INTO Students (FirstName, Email) VALUES ('Raj', NULL);

-- Empty String = Zero-length string
INSERT INTO Students (FirstName, Email) VALUES ('Raj', '');

-- To check for NULL, use IS NULL (not = NULL)
SELECT * FROM Students WHERE Email IS NULL; -- Correct
SELECT * FROM Students WHERE Email = NULL; -- Won't work
```

---

### Q4: Explain the order of execution in SQL query?

**Answer:** Even though you write SQL in this order:

SELECT → FROM → WHERE → ORDER BY

**SQL executes in this order:** 1. **FROM** - Identify the table 2. **WHERE** - Filter rows 3. **SELECT** - Choose columns 4. **ORDER BY** - Sort results

**Example:**

```
SELECT FirstName, GPA -- 3. Choose columns
FROM Students         -- 1. From this table
WHERE GPA > 3.5        -- 2. Filter rows
ORDER BY GPA DESC;    -- 4. Sort results
```

---

### Q5: What is the difference between WHERE and HAVING?

**Answer:** | Aspect | WHERE | HAVING | |---|---|---| | **Used with**  
 | Individual rows | Grouped data (aggregate functions) | | **Placement** |  
 Before GROUP BY | After GROUP BY | | **Example** | WHERE Age > 20 |  
 HAVING COUNT(\*) > 5 |

```
-- WHERE - Filter rows before grouping
SELECT Department, COUNT(*) AS Count
FROM Employees
WHERE Salary > 50000 -- WHERE filters rows
GROUP BY Department;

-- HAVING - Filter groups after grouping
SELECT Department, COUNT(*) AS Count
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 5; -- HAVING filters groups
```

---

### Q6: Can you use ORDER BY on a column that's not in SELECT?

**Answer: Yes!** You can sort by any column from the table, even if it's not displayed.

```
SELECT FirstName FROM Students ORDER BY GPA DESC;
-- This is valid - sorts by GPA but only shows FirstName
```

---

### Q7: What happens if you run UPDATE without WHERE clause?

**Answer: All rows will be updated!** This is dangerous.

```
UPDATE Students SET GPA = 4.0;
-- ⚠ All students' GPA becomes 4.0!

-- Correct way:
UPDATE Students SET GPA = 4.0 WHERE FirstName = 'Raj';
-- Only Raj's GPA is updated
```

---

### Q8: Explain UNIQUE constraint vs PRIMARY KEY?

**Answer:** | Aspect | UNIQUE | PRIMARY KEY | |---|---|---| |  
**NULL values** | Multiple NULLs allowed | Only one NULL (or none) | |  
**Count** | Multiple per table | Only one per table | | **Uniqueness** |  
Ensures unique values | Unique identifier | | **Usage** | Email, Phone | ID |

```
CREATE TABLE Users (
    UserID INT PRIMARY KEY,      -- One PK
    Email VARCHAR(100) UNIQUE,  -- Multiple UNIQUE allowed
    Phone VARCHAR(20) UNIQUE
);
```

---

### Q9: What is IDENTITY and how does it work?

**Answer: IDENTITY** auto-generates unique numbers for a column (usually for PRIMARY KEY).

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY IDENTITY(1,1), -- Start at 1,
increment by 1
    FirstName NVARCHAR(50)
);

-- When you insert:
INSERT INTO Students (FirstName) VALUES ('Raj');
-- StudentID is automatically set to 1

INSERT INTO Students (FirstName) VALUES ('Priya');
-- StudentID is automatically set to 2
```

---

### Q10: How do you view all tables in a database?

**Answer:**

```
-- Method 1: Query system views
SELECT * FROM INFORMATION_SCHEMA.TABLES;

-- Method 2: Using sp_tables stored procedure
EXEC sp_tables;
```

-- Method 3: In SQL Server Management Studio  
-- View > Object Explorer > Databases > YourDB > Tables

---

## Summary Cheat Sheet

```
-- CREATE TABLE
CREATE TABLE Students (
    StudentID INT PRIMARY KEY IDENTITY(1,1),
    FirstName NVARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    GPA DECIMAL(3,2),
    IsActive BIT DEFAULT 1
);

-- INSERT
INSERT INTO Students (FirstName, Email, GPA)
VALUES ('Raj', 'raj@email.com', 3.8);

-- SELECT
SELECT FirstName, Email FROM Students;

-- WHERE
SELECT * FROM Students WHERE GPA > 3.5;

-- ORDER BY
SELECT * FROM Students ORDER BY GPA DESC;

-- UPDATE
UPDATE Students SET GPA = 3.9 WHERE StudentID = 1;

-- DELETE
DELETE FROM Students WHERE StudentID = 5;
```

---

## Key Takeaways

1. **SQL is case-insensitive** (SELECT = select)
  2. **Always use WHERE when updating/deleting** to avoid accidents
  3. **Use single quotes** for string values: 'value'
  4. **NULL is not the same as empty string**
  5. **ORDER BY can use columns not in SELECT**
  6. **Test SELECT before DELETE/UPDATE** on the same WHERE clause
  7. **PRIMARY KEY ensures data integrity and uniqueness**
  8. **IDENTITY auto-generates unique numbers**
- 

## Next Steps

- Practice the exercises multiple times
- Create your own tables and practice CRUD operations
- Understand table relationships (covered in Day 3-4: Joins)
- Get comfortable with SQL syntax before moving to Joins

**Happy Coding!**